Tool: Prosecute Your Paper To Improve It

Document Number:	P4170R0
Date:	2026-03-30
Audience:	WG21

Reply-to:	Vinnie Falco vinnie.falco@gmail.com

Table of Contents

Abstract

For less than a dollar, every paper in the mailing can know its own weaknesses before the committee does.

AI-powered adversarial analysis of WG21 papers now costs under a dollar and takes minutes. This paper introduces the Advocatus Diaboli (Devil's Advocate), a purpose-built red-teaming methodology for committee papers, and demonstrates it on P2900R14[1], "Contracts for C++." The case study produced five sections certified as battle-hardened, five formal objections from nine candidates after adversarial cross-examination, and sixteen falsifiable predictions with one-, two-, and three-year horizons. The predictions form a retrospective framework - the accountability mechanism the committee does not yet require and this tool provides for itself. The methodology, the case study, and the retrospective are presented here so the reader can judge the tool by its output.

Revision History

R0: April 2026 (post-Croydon mailing)

- Initial version.

---

1. Disclosure

The author provides information and serves at the pleasure of the committee.

The author is the founder of the C++ Alliance. The author maintains competing proposals in the `std::execution` space and is a co-author of P4003R0[2], P4007R0[3], and P4100R0[4]. The author has no competing Contracts proposal. P2900R14[1] is used as a case study because it is large, consequential, and well-documented - not because the author has a stake in its outcome.

This paper uses AI. The case study in Section 6 was produced by an AI tool. The predictions in Section 7 were generated by the same AI tool. This paper promotes the use of AI to evaluate committee papers before submission. The author believes this use of AI improves the quality of the committee's work product.

The AI analysis was performed using a structured prompt - the Advocatus - against the publicly available text of P2900R14[1] and its companion rationale paper P2899R1[5]. The full methodology is introduced in Section 4.

This paper is a companion to P4133R0[6], "What Every Proposal Must Contain," which defines what a healthy feedback loop would contain for library and language proposals. That paper asks whether the committee evaluates its own output. This paper provides a tool that begins the evaluation before the committee sees the paper.

The committee decides. Consensus must be accepted even when the author disagrees with the outcome. The author's position is that P2900 should not have been considered for wording until the proposers had presented a complete working implementation deployed into an organization, with independent adoption sufficient to gather user experience and feedback. A paper's evaluation by the committee should begin when user feedback arrives, not before. Instead, with Contracts, user feedback will arrive after the standard ships.

Contracts shipped to the standard before they shipped to regular users.

The author asks for nothing.

---

2. Human Judgment Is Required at Every Step

When interpreting the output of the Advocatus or any AI tool that performs adversarial evaluation of a paper, the human must evaluate each finding and determine whether it is valid. The AI identifies candidate weaknesses. The human decides which candidates are real.

The AI does not know what the author knows. The AI does not know what the committee discussed last Tuesday. The AI does not know that a seemingly devastating objection was already resolved in a private conversation, or that a stakeholder changed position after publication. Only the human knows these things. Without human judgment, the output is suspect and must not be assumed correct. The tool produces leads. The human produces conclusions.

The Advocatus is designed to make this collaboration easier. The Interrogatory (the questioning phase) deposes the author with multiple-choice questions before any charge is filed. The Advocatus Dei (defender of the cause) cross-examines every candidate finding before it reaches the record. But the design does not eliminate the requirement for human judgment. It structures the conversation between the human and the machine so that the human's knowledge enters the process at the right moments.

This principle applies to all AI usage, not only adversarial paper prosecution. Every AI output - code review, specification drafting, evidence gathering - requires human judgment before it becomes actionable. The human is the quality control. The AI is the search engine.

---

3. The Cost of Not Looking

The committee has no systematic pre-submission red-teaming practice. Authors self-review. Reviewers are busy. The mailing contains hundreds of papers per cycle. The result: technical weaknesses, citation errors, and political vulnerabilities arrive at the committee table for the first time during presentation.

The cost of this gap is not hypothetical. C++20 Contracts were adopted at Cologne in 2019 and removed at Prague in 2020[7]. The cost of that cycle was measured in years of committee time, implementation effort, and community trust. P2900R14[1] - the second attempt - was adopted into the C++26 working draft at Hagenberg in February 2025. Multiple national bodies subsequently filed substantive objections[8]. Each NB comment requires committee time to process.

AI has changed the economics. What once required a team of domain experts and weeks of reading can now be approximated in minutes for pennies. The analysis in Section 6 cost less than one dollar in API tokens. The question is no longer whether pre-submission red-teaming is worth the cost. The question is whether there is any remaining excuse not to do it.

---

4. Introducing the Examiners

The examiner is a structured prompt - a set of instructions that a frontier language model follows. It is open source, hosted at the C++ Alliance, and available in three cultural translations. All three implement the same twenty rules, the same five phases, the same six counter-examination challenges. The operational logic is identical. The metaphor, tone, and terminology change to suit the reader's cultural expectations. Copy the prompt into any frontier model (Claude, GPT, Gemini) and point it at a paper. Each tool is dedicated to the public domain under CC0 1.0 Universal[11].

4.1 Advocatus Diaboli

Advocatus Diaboli is a structured red-teaming tool for WG21 (C++ committee) papers. It models itself on the historical Catholic office of the Devil's Advocate - the appointed adversary whose job was to challenge candidates for sainthood before canonization.

The tool takes a WG21 paper as input and subjects it to a formal five-phase adversarial examination:

1. **Vocatio (The Summons)** - Classifies the paper (ask vs. inform), determines posture (defending your own paper vs. scouting an opponent's), and reads the paper to extract every claim it actually makes.
2. **Inquisitio (The Investigation)** - Assembles a dossier via web searches, MCP queries, workspace intelligence, and stakeholder analysis. Verifies all citations. Delegates research to sub-agents to preserve the main context window for judgment.
3. **Interrogatorium (The Interrogatory)** - Deposes the user (the "postulator") with structured questions to resolve assumptions the dossier could not confirm.
4. **Examen (The Examination)** - Tests every claim against three criteria: factual accuracy (Veritas), logical soundness (Ratio), and citation support (Auctoritas). Candidate objections are then cross-examined by an internal "Advocatus Dei" (defender) through six challenges that filter out noise - confessions, phantom claims, things better asked than charged, non-human objections, self-defeating arguments, and trivial housekeeping.
5. **Animadversiones (The Observations)** - Delivers the formal verdict: ***nihil obstat*** (nothing stands in the way), ***cum obiectionibus*** (with objections), or ***suspendatur*** (suspended pending testimony). Output includes imprimaturs for battle-hardened sections, surviving objections with named adversaries and damage assessments, minor notes, an audit trail, and a citation verification table.

Key design principles:

- **The Oath**: The tool is designed to ***prefer*** finding nothing wrong. An advocate that always produces findings is performing theater, not analysis. ***Nihil obstat*** is the highest outcome, not a failure state.
- **Posture-aware**: If it is your paper, findings say "fix this." If it is someone else's, findings say "press here."
- **Boundary discipline**: It only examines claims the paper actually makes - never attacks what the paper did not say.
- **Token discipline**: Research is delegated to sub-agents; the main context window is reserved for judgment. While sub-agents work, the tool emits dispatches styled as reports from Roman-named medieval archivists.
- **Iterative convergence**: On re-examination after revision, prior findings carry forward and the scope narrows toward either resolution or accepted trade-offs.

The Advocatus Diaboli ("Devil's Advocate")

4.2 Der Werkprüfer

Der Werkprüfer is a structured inspection framework for WG21 (C++ committee) papers. It models itself on the German TÜV inspection system - the appointed Werkprüfer whose job was to inspect the Werkstück (workpiece) before it reached the Prüfungsausschuss (inspection committee). A built-in adversarial counterpart - the Werkmeister (defender of the craft) - challenges every candidate defect before it can enter the report.

The tool takes a WG21 paper as input and subjects it to a formal five-phase inspection:

1. **Einberufung (The Convocation)** - Classifies the paper (ask vs. inform), determines posture (hardening your own paper vs. scouting an opponent's), and reads the paper to extract every claim (Prüfpunkte) it actually makes through four readings.

2. **Ermittlung (The Investigation)** - Assembles an evidence file (Aktenmappe) via web searches, MCP queries, workspace intelligence, and stakeholder analysis. Verifies all citations through a three-pass cascade. Delegates research to sub-agents to preserve the main context window for judgment.

3. **Befragung (The Interrogation)** - Deposes the user (the Geselle) with structured questions to resolve assumptions the Aktenmappe could not confirm. Each answer becomes established fact that shapes every subsequent finding.

4. **Prüfung (The Examination)** - Tests every claim against three criteria: factual correctness (Sachrichtigkeit), logical coherence (Schlüssigkeit), and citation support (Quellenbeleg). Candidate defects (Mängel) are then cross-examined by the Werkmeister (defender) through six challenges that filter out noise - concessions (Zugeständnis), phantom claims (Grenzüberschreitung), things better asked than charged (Rückfrage), non-human objections (Menschenmaß), self-defeating arguments (Selbstschaden), and trivial housekeeping (Bagatelle). Survivors get a named adversary, forum, and damage assessment (Begründung).

5. **Prüfbericht (The Report)** - Delivers the formal verdict (Prüfsiegel): ***Freigabe*** (released for use), ***Nachbesserung erforderlich*** (rework required), or ***Prüfung ausgesetzt*** (inspection suspended pending testimony). Output includes Freigabe certifications for battle-hardened sections, surviving defects with named adversaries and damage assessments, minor notes (Randnotizen), an audit trail (Aktenzusammenfassung), and a citation verification table (Quellenverzeichnis).

Key design principles:

- **The Oath (Prüfereid)**: The tool is designed to ***prefer*** finding nothing wrong. An inspector who always finds defects has already broken the instrument. ***Freigabe*** is the highest outcome, not a failure state.
- **Posture-aware**: If it is your paper, the tool operates in ***Härtung*** (hardening) mode - findings say "correct this." If it is someone else's, the tool operates in ***Erkundung*** (reconnaissance) mode - findings say "press here."
- **Boundary discipline**: It only examines claims the paper actually makes (the Prüfpunkte) - never attacks what the paper did not say. The fourth reading explicitly maps what the paper disclaims, concedes, and

leaves to the reader.

- **Token discipline**: Research is delegated to sub-agents; the main context window is reserved for judgment. While sub-agents work, the tool emits dispatches styled as status updates from fictional German-named lab engineers in a Prüflabor.
- **Iterative convergence**: On re-inspection (Nachprüfung) after revision, prior findings carry forward and the scope narrows toward either resolution or accepted trade-offs.

Der Werkprüfer (“The Workpiece Inspector”)

4.3 The Shenyueguan

The Shenyueguan (□阅官, reviewing official, appointed examiner) is a structured review framework for WG21 (C++ committee) papers. It models itself on the Chinese imperial court examination system - the appointed □阅官 whose job was to examine the 呈文 (submission) before it reached the □文院 (review court). A built-in adversarial counterpart - the 护文官 (document-defending official) - sits opposite the examiner and challenges every candidate finding before it can enter the record.

The tool takes a WG21 paper as input and subjects it to a formal five-phase adversarial examination:

1. □集 **(The Convocation)** - Classifies the paper (ask vs. inform), determines posture (polishing your own paper vs. probing an opponent’s), and reads the paper to extract every claim (论点) it actually makes through four readings.

2. 探查 **(The Investigation)** - Assembles a case dossier (案卷) via web searches, MCP queries, workspace intelligence (内部情报), and stakeholder analysis. Verifies all citations through a three-pass cascade. Delegates research to sub-agents to preserve the main context window for judgment.

3. 问询 **(The Interrogation)** - Deposes the user (the 呈文人) with structured questions to resolve assumptions the 案卷 could not confirm. Each answer becomes 定论 (established fact) that shapes every subsequent finding.

4. 审阅 **(The Examination)** - Tests every claim against three criteria: factual accuracy (真伪), logical soundness (理路), and citation support (出处). Candidate objections (驳) are then cross-examined by the 护文官 (defender) through six challenges that filter out noise - concessions (已认), phantom claims (边界), things better asked than charged (待问), non-human objections (人情), self-defeating arguments (自伤), and trivial housekeeping (细末). Survivors get a named adversary, forum, and damage assessment (判决理由).

5. 审文录 **(The Review Record)** - Delivers the formal verdict (□文印): □予通行 (approved to proceed), 驳回待改 (returned for revision), or 缓议 (deferred pending testimony). Output includes 准 certifications for battle-hardened sections, surviving objections with named adversaries and damage assessments, minor notes (附注), an audit trail (案卷总结), and a citation verification table (溯源表).

Key design principles:

- **The Oath (□□□)**: The tool is designed to ***prefer*** finding nothing wrong. An examiner who always finds fault has already lost the court's trust. □□□□ is the highest outcome, not a failure state.
- **Posture-aware**: If it is your paper, the tool operates in □□ (polishing) mode - findings say "correct this." If it is someone else's, the tool operates in □□ (case probing) mode - findings say "press here."
- **Boundary discipline**: It only examines claims the paper actually makes (the □□) - never attacks what the paper did not say. The fourth reading explicitly maps what the paper disclaims, concedes, and leaves to the reader.
- **Token discipline**: Research is delegated to sub-agents; the main context window is reserved for judgment. While sub-agents work, the tool emits dispatches styled as reports from court runners (□□) with classical Chinese names returning through the court gate.
- **Iterative convergence**: On re-examination (□□) after revision, prior findings carry forward and the scope narrows toward either resolution or accepted trade-offs.

The Shenyueguan ("The Appointed Examiner")

---

5. Why Not Just Ask the AI?

The obvious alternative: paste a paper into any frontier language model and say "find the weaknesses." The result is a flat list of concerns - some real, some phantoms - with no internal quality control. Every observation has equal weight. The model has no mechanism to distinguish a devastating objection from a formatting quibble, no way to test whether the paper already concedes the point, and no adversarial cross-examination of its own output. The author receives a brainstorm, not a briefing, and must do the analysis the tool was supposed to do for them.

The examiner is a purpose-built methodology for WG21 papers. Five differences matter.

5.1 The Counter-Examiner Kills Bad Findings

The counter-examiner (the Advocatus Dei, the Werkmeister, the □□□) sits opposite the examiner. Every candidate charge must survive six named challenges before it reaches the record:

- **Confessio.** Does the paper already concede this point? A charge that attacks a concession is theater.
- **Articulus.** Does the paper actually claim what the objection attacks? A charge that attacks an inference the examiner drew, rather than a claim the paper stated, is a phantom.
- **Testimonium.** Could a single question to the author dissolve this? If a ten-second answer would collapse the charge, it should have been a question, not a finding.

- **Humanitas.** Would a real human opponent make this argument? If the objection exists only because a machine performed exhaustive analysis no committee member would replicate, it is suppressed.
- **Prudentia.** Would making this argument be self-defeating for the actual opponent? If pressing the objection requires the named adversary to undermine their own published position, no rational adversary would volunteer it.
- **Dignitas.** Is this objection beneath the dignity of the office? Typos, formatting, word-choice quibbles - these are housekeeping, not charges.

In the P2900 case study, 4 of 9 candidate charges were killed by the counter-examiner. A generic red team would have reported all 9.

5.2 The Tool Asks Instead of Guessing

The questioning phase deposes the author before any charge is filed. The tool asks multiple-choice questions to establish ground truth: who are the real opponents, what is the paper's strategic objective, what does the political landscape look like. Generic red-teaming guesses at context. The examiner asks. The author's answers become sworn testimony that shapes every subsequent finding.

Every surviving charge must name a specific adversary, a specific forum, and a specific damage assessment. Not "a careful reader might object" but "Eric Fiselier, in an NB comment, arguing that the mixed-mode model makes the feature unusable for library code." The difference between a list of concerns and an actionable briefing is the difference between "someone might not like this" and "here is who, here is where, here is the damage."

5.3 The Tool Clears Papers and Certifies Strengths

Three seals give the author a verdict, not a list. The tool can clear a paper entirely. A generic red team never clears anything; it always finds something, because finding nothing feels like a failure. The examiner that always finds fault has already failed - the Catholic Church proved this in 1983 when it dissolved the Advocatus Diaboli office and canonizations accelerated twentyfold with no one left to say no.

Sections that withstand opposition are certified. No generic red team tells the author "do not engage here - their defense holds." The positive signal is as valuable as the negative one. Knowing where the paper is strong saves the author from wasting revisions on sections that need no revision.

5.4 The Tool Is Deliberately Entertaining

The medieval tribunal metaphor - the Vocatio (summons), the Inquisitio (investigation), the Animadversiones (formal observations), the seals in Latin - is not decoration. It is an adoption strategy.

AI analysis takes time. The author waits while the tool searches, cross-references, and deliberates. A tool that is boring during those minutes is a tool the author uses once and forgets. A tool that is interesting - that has a narrative arc, that renders judgment with ceremony, that feels like an event rather than a chore - is a tool the author runs on every revision. The entertainment value is load-bearing. Better papers are written by authors who actually use the tool, and authors use tools they enjoy.

5.5 The Form Is Not the Point

After Croydon, I presented the Advocatus to a German colleague. He thought it was frivolous. Too much ceremony, too much Latin, too much narrative dressing on what should be a straightforward technical process. He did not see the value in the metaphor. He wanted the function without the theater.

I took this to heart. He was not wrong - he was German. His culture values directness, technical precision, and Ordnung. The ecclesiastical framing that I found motivating, he found obstructive. The Latin terminology that I thought added gravity, he thought added fluff. The same methodology, presented the same way, produced opposite reactions depending on which side of the Rhine the reader grew up on.

So I translated it. Der Werkprüfer - the workpiece inspector - is the same tool rewritten for a culture that prefers calipers to crucifixes. The paper becomes a Werkstück (workpiece). The author becomes a Geselle (journeyman). The tribunal becomes a Prüfkammer (inspection chamber). The seals are Freigabe (released), Nachbesserung erforderlich (rework required), and Prüfung ausgesetzt (inspection suspended). The dispatches come from German-named engineers in a Prüflabor, not Roman-named archivists in a scriptorium. Every Latin term becomes German. The tone is terse, direct, and unapologetically technical.

The operational logic is identical. The same twenty rules. The same sub-agent delegation. The same six Werkmeister challenges (the Advocatus Dei, now speaking German). The same three tests per claim. The same output structure. The methodology did not change. The metaphor changed. The function survived the translation intact.

This is the deeper point, and it is not limited to Germans. WG21 is an international committee. Japanese delegates, Indian delegates, Finnish delegates, Brazilian delegates - each brings cultural expectations about how technical analysis should be framed, how authority should be expressed, how criticism should be delivered. A tool that assumes every user shares the author's cultural preferences is a tool that excludes everyone who does not. Customizability is not a convenience. It is a requirement.

The examiner is a prompt - a structured set of instructions that an LLM follows. LLMs are transformers. They translate. If the medieval tribunal metaphor does not suit the reader, strip it. Replace it with a German engineering inspection, a courtroom cross-examination, a peer review committee, or no metaphor at all. The twenty rules and five phases work without any framing at all. The form is a vehicle for adoption. The function is the methodology itself. Do not confuse them.

The remainder of this paper provides a worked example.

6. Case Study: P2900R14, "Contracts for C++"

P2900R14[1] is a significant achievement. Fourteen revisions over three years. Genuine consensus in SG21 on questions where consensus seemed unlikely. A clean syntax (`pre` , `post` , `contract_assert`) that survived multiple alternative explorations. Sixteen design principles, internally consistent and clearly stated. A violation handler model that mirrors the replaceable `operator new` pattern C++ programmers already understand. CWG and LWG wording review completed. Two independent compiler implementations (GCC and Clang) available on Compiler Explorer.

The Advocatus examines it not to diminish it but because the best papers deserve the hardest test. The following are the formal observations (***Animadversiones***) produced by the Advocatus, quoted from the analysis output.

6.1 The Seal

Cum obiectionibus. The cause proceeds with five objections.

6.2 Sections Certified as Battle-Hardened (*Imprimatur*)

Design Principles (Section 3.1): ***Imprimatur.*** The sixteen design principles are internally consistent, clearly stated, and well-grounded. The Prime Directive, the Redundancy Principle, and the Zero Overhead principle form a coherent foundation. The Advocatus Dei prevailed on the Humanitas challenge - no committee member would dispute these principles in isolation. Do not engage here; the defense holds.

Syntax (Section 3.2): ***Imprimatur.*** The `pre` / `post` / `contract_assert` syntax is clean, already implemented in two compilers, and has survived multiple SG21 votes. The Advocatus Dei prevailed on the Confessio challenge - the rationale paper P2899R1[5] documents the extensive alternative syntax exploration openly. No remaining attack surface on syntax. Do not engage here.

Semantic Restrictions on Constructors/Destructors (Section 3.3.4): ***Imprimatur.*** The rule making direct nonstatic data member access ill-formed in constructor preconditions and destructor postconditions is a precise, defensible design choice. The Advocatus Dei prevailed on the Prudentia challenge - attacking this restriction would require arguing for either weaker safety or stronger restriction, neither of which an opponent would volunteer.

Constant Evaluation Rules (Section 3.5.12): ***Imprimatur.*** The trial evaluation mechanism is technically sound and respects Design Principle 4 (Zero Overhead). The Advocatus Dei prevailed on the Humanitas challenge - the constant evaluation rules are sufficiently arcane that no committee member would mount a floor attack here.

Standard Library API (Section 3.7): ***Imprimatur.*** The `<contracts>` header design is conservative and extensible. The Advocatus Dei prevailed on the Dignitas challenge - any attack on library API details is editorial, not substantive.

6.3 Formal Objections

Objection I: The Implementation-Defined Evaluation Semantic Selection Is a Design Defect, Not a Feature. (Severity: Critical. Test failed: Ratio.) P2900 delegates the most consequential runtime behavior decision - whether a contract assertion does nothing, observes, enforces, or crashes - entirely to implementation-defined mechanisms. The paper's own Principle 5 (Independence from Chosen Semantic) acknowledges the tension. The consequence is that the same source code compiled by different vendors produces programs with fundamentally different safety properties, and the programmer has no portable mechanism to express intent. P3835R0[9] (Spicer, Voutilainen, Garcia Sanchez) demonstrates the concrete consequence: when library headers compiled with one semantic are inlined into client code compiled with another, the effective semantic is determined by optimization decisions, not programmer intent.

Objection II: The Deployment Experience Claim Does Not Withstand Scrutiny. (Severity: Critical. Test failed: Veritas.) The paper claims "deployment experience" but the evidence consists of two controlled experiments by Contracts proponents - replacing existing assertion macros in LLVM and libc++ with P2900 contract assertions. No production codebase has been reported as using P2900's novel features (`pre` / `post` on function declarations, the violation handler replacement mechanism, multiple evaluation semantics in a mixed TU environment) at scale. P4020R0[10] (Krzemienski - a P2900 co-author) states explicitly that the committee's experience is limited to assertion statements in function bodies, not declaration annotations.

Objection III: Predicate Side-Effect Elision Creates an Unspecified Behavior Surface. (Severity: Significant. Test failed: Ratio.) The combined effect of predicate side-effect elision (Section 3.5.8) and evaluation repetition (Section 3.5.7) means that side effects in predicates may occur zero, one, or an implementation-defined number of times. For a feature marketed on safety and correctness, introducing a new class of unspecified-count-of-execution is a tension in the narrative.

Objection IV: The Mixed-Mode Compilation Model Is Novel, Untested, and Produces Semantic Non-Determinism. (Severity: Significant. Test failed: Ratio and Veritas.) Section 3.5.11 acknowledges that when inline functions are compiled in different translation units with different evaluation semantics, the linker selects "effectively random" which definition survives. The paper's own text uses those words to describe the behavior of a safety feature.

Objection V: Six Instances of Implementation-Defined Behavior Exceed the Threshold for a First-Version Feature. (Severity: Moderate. Test failed: Ratio.) P2899R1[5] Section 2.10 lists six categories of implementation-defined behavior: semantic selection, evaluation count, termination mode, default handler behavior, handler

replaceability, and `contract_violation` polymorphism. The defense characterizes these as "well-known cases for which no single answer is a viable solution." That characterization is precisely the argument for a TS rather than direct IS adoption.

6.4 Minor Observations (*Notae Minores*)

1. Making `pre` / `post` on virtual functions ill-formed (Section 3.3.2) is a significant functional limitation that the paper concedes openly.
2. The `contract_assert` keyword is unavoidably long and will resist adoption by developers accustomed to `assert`.
3. P2899R1[5] Section 4.1 documents an errata in P2900R14's Section 3.5.3 that "cannot be corrected directly in that paper because it has already been approved by WG21 plenary."
4. Postconditions on coroutines cannot odr-use parameters - a consequence of the coroutine parameter copy model, not a Contracts design flaw.

6.5 The Acta (Audit Trail)

The Advocatus investigated the full text of P2900R14 (256KB, 4348 lines), the full text of P2899R1 (419KB, 6256 lines), the public record (web search), indexed archives, and local workspace files. The author was deposed with three sets of multiple-choice questions establishing posture, strategic objective, and deployment experience weight. Nine candidate charges were filed. Four were killed by the Advocatus Dei (Confessio: 2, Articulus: 1, Humanitas: 1). Five survived to the formal observations. The analysis cost less than one dollar in API tokens and took approximately fifteen minutes of wall-clock time.

---

7. The Missing Retrospective

P4133R0[6] documented a generic gap: the committee does not require retrospective frameworks for adopted features. No testable predictions. No falsification criteria. No timeline for measuring success or failure. P2900 shipped without one. Every large feature ships without one.

The retrospective below exists to test the Advocatus's predictive value. Every prediction is a direct consequence of a specific finding from the case study in Section 6. Each prediction names its source. In one, two, or three years, the committee can return to this list and measure how many predictions materialized. If most did, the tool has predictive value and the case for pre-submission red-teaming strengthens. If most did not, the tool's judgment on P2900 was wrong, and that is worth knowing too. The retrospective is the accountability mechanism the tool provides for itself.

Only predictions that trace directly to a finding are included. Predictions about vendor behavior, political dynamics, or feature evolution that go beyond the five objections and five imprimaturs have been removed. The retrospective tests the findings, not the author's speculation.

7.1 Failure-Mode Predictions

- **No organization unaffiliated with P2900's co-authors reports production deployment of declaration-level annotations within two years** (Objection II). Falsified by: two or more unaffiliated organizations reporting production `pre` / `post` use.
- **At least two compiler vendors ship incompatible default evaluation semantics** (1 year, Objection I). Falsified by: all three major compilers shipping identical defaults.
- **A paper or DR addresses the mixed-mode "effectively random" inline function semantic selection** (1 year, Objection IV). Falsified by: no such paper filed through the 2027 mailing cycle.
- **The predicate side-effect elision/repetition model produces at least one public bug report where a diagnostic side effect was unexpectedly elided or repeated** (1 year, Objection III). Falsified by: no such report filed against GCC or Clang.
- **The P2899 errata (Section 4.1) produces a formal DR** (1 year, Nota Minor 3). Falsified by: no DR filed addressing the Observable Checkpoints errata.
- **Header-only libraries avoid `pre` / `post` in public headers because authors cannot control the client's evaluation semantic** (2 years, Objection IV). Falsified by: three or more popular header-only libraries shipping `pre` / `post` in public headers.
- **A portable semantic-binding mechanism is proposed** (1 year) **and adopted** (3 years, Objection I). Falsified by: no such paper in 2027; no adoption by end of 2029.
- **The Standard Library (libstdc++, libc++, MSVC STL) does not add `pre` / `post` annotations to any function declarations** (2 years, Objection II). Falsified by: any major stdlib shipping declaration annotations on a public header function.
- **The violation handler replacement mechanism produces at least one public report of unexpected behavior across shared library boundaries** (2 years, Objection V). Falsified by: no such report through end of 2028.

7.2 Success-Mode Predictions

- **`contract_assert` replaces `assert` in at least five major codebases' coding standards** (2 years, Syntax imprimatur). Falsified by: fewer than five making the switch.

- **At least one major static analysis tool consumes `pre` / `post` annotations for dataflow analysis, even when runtime checking is disabled** (3 years, Design principles imprimatur). Falsified by: no major tool consuming annotations by end of 2029.
- **At least two large organizations publicly report integrating the replaceable violation handler with production crash-reporting infrastructure** (3 years, Library API imprimatur). Falsified by: no organization reporting such integration.
- **At least one published report presents measured bug-density reduction attributable to adopting `pre` on public API functions** (3 years, Objection II inverse). Falsified by: no such report by end of 2029.
- **At least one safety-related standards body or regulatory document cites C++26 Contracts as evidence of C++ addressing correctness** (2 years, Design principles imprimatur). Falsified by: no such citation by end of 2028.
- **At least one major C++ textbook incorporates `pre` / `post` syntax as the primary way to express function contracts** (3 years, Syntax imprimatur). Falsified by: no such textbook adoption by end of 2029.

---

8. The Economics

The P2900 analysis produced:

- 5 imprimaturs (sections certified as battle-hardened)
- 5 formal objections (from 9 candidates after Dei cross-examination)
- 4 notae minores (editorial observations)
- 15 falsifiable predictions with explicit timelines

The cost:

- API tokens: under $1
- Wall-clock time: approximately 15 minutes
- Human effort: answering 3 sets of multiple-choice questions

Compare to the cost of not running the analysis. C++20 Contracts were adopted at Cologne in 2019 and removed at Prague in 2020[7]. The cost of that single cycle was years of committee time, implementation effort in two compilers, and community trust that has not fully recovered six years later. P2900's post-adoption NB comments[8] - each requiring committee time to process - raised objections the Advocatus identified in fifteen minutes.

A dollar and fifteen minutes. That is the new cost of knowing a paper's weaknesses before the committee does.

---

9. Conclusion

Contracts is not the target of this paper. The case study is Contracts because someone asked me to run the analysis and Contracts happened to be the paper on the table. The same tool, applied to any feature of comparable scope - Ranges, Modules, `std::execution`, including my own proposals - would produce findings. Every large feature has weaknesses. The tool finds them. The feature that happened to be on the table is not at fault for being on the table.

The Contracts authors did years of careful, good-faith work. Fourteen revisions. Genuine consensus on hard questions. The findings in this paper are not a failure of the authors. They are a symptom of a process that evaluates proposals before users have touched them. We have gotten comfortable with the process - meetings held, polls taken, standards shipped on schedule - and stopped paying as much attention to outcomes. Did the feature achieve what it claimed? Did the users adopt it? Did the predictions hold? We measure whether the train departed. We do not measure whether the cargo was worth shipping.

Members need to be more demanding. Delegates need to be more demanding. Chairs need to be more demanding. The convener needs to be more demanding. As engineers, we need to be more demanding. Users need to be more demanding. Not demanding in the sense of raising the bar to block progress - demanding in the sense of requiring evidence over enthusiasm. An abundance of evidence. Implementation, deployment, user feedback, independent adoption, measured outcomes. The tools to generate that evidence now cost a dollar. The discipline to require it costs nothing at all.

The best version of every paper already exists. It is the version the author writes after seeing what the opposition will say.

Acknowledgements

Joshua Berne, Timur Doumler, and Andrzej Krzemieński, for producing a paper substantial enough to serve as a meaningful case study. Fourteen revisions is not a weakness. It is a measure of the seriousness with which the authors treated the problem.

The author's experience meeting people of German background at Croydon, who helped the author understand that the tool needs to accommodate cultural differences.

Anthropic, for Claude - the model that powered the analysis.

References

1. P2900R14 - "Contracts for C++" (Joshua Berne, Timur Doumler, Andrzej Krzemieński, 2025). https://wg21.link/p2900r14
2. P4003R0 - "Coroutines for I/O" (Vinnie Falco, Mungo Gill, Steve Gerbino, 2026). https://wg21.link/p4003r0
3. P4007R0 - "Senders and Coroutines" (Vinnie Falco, Mungo Gill, 2026). https://wg21.link/p4007r0
4. P4100R0 - "The Network Endeavor" (Vinnie Falco, Mungo Gill, 2026). Post-Croydon mailing.
5. P2899R1 - "Contracts for C++ - Rationale" (Joshua Berne, Timur Doumler, Andrzej Krzemieński, Rostislav Khlebnikov, 2025). https://wg21.link/p2899r1
6. P4133R0 - "What Every Proposal Must Contain" (Vinnie Falco, 2026). Post-Croydon mailing.
7. P0542R5 - "Support for contract based programming in C++" (G. Dos Reis, J. D. Garcia, J. Lakos, A. Meredith, N. Myers, B. Stroustrup, 2018). https://wg21.link/p0542r5. Adopted at Cologne 2019, removed at Prague 2020.
8. P3846R0 - "C++26 Contracts, reasserted" (Timur Doumler, Joshua Berne et al., 2025). https://wg21.link/p3846r0. Responds to NB comments filed post-adoption.
9. P3835R0 - "Contracts make C++ less safe - full stop" (John Spicer, Ville Voutilainen, Jose Daniel Garcia Sanchez, 2025). https://wg21.link/p3835r0
10. P4020R0 - "Concerns about contract assertions" (Andrzej Krzemieński, 2026). https://wg21.link/p4020r0
11. CC0 1.0 Universal - Creative Commons Public Domain Dedication. https://creativecommons.org/publicdomain/zero/1.0/